

1. Logging & Tracing (Agent Steps)

- **The Problem:** When an agent fails, "the LLM output was bad" isn't a good enough answer. You need to know *which* step failed. Did the tool return bad data, or did the LLM misinterpret good data?
- **The Solution:** Implement **Trace IDs** (using tools like LangSmith, Arize Phoenix, or OpenTelemetry). You should be able to see a "Tree View" of the execution:
 - *Level 1:* User Query
 - *Level 2:* LLM Reasoner (Thought process)
 - *Level 3:* Tool Call (API Request/Response)
 - *Level 4:* Final Synthesis.
- **Interview Signal:** "I don't just log inputs/outputs; I log the intermediate thoughts and tool metadata to debug the reasoning chain."

2. Prompt Versioning

- **The Problem:** Changing one word in a prompt to fix a bug in "Meal Planning" might accidentally break "Grocery List" generation.
- **The Solution:** Treat prompts like code. Use a **Prompt Management System** (like Pezzo or Portkey) or a Git-based workflow.
- **The Strategy:** Never hard-code prompts. Pull them from an API or a versioned config file. Implement **A/B Testing**—run 10% of traffic on "v2.1-optimized" to see if it actually performs better before a full rollout.

3. Cost Monitoring

- **The Problem:** An autonomous agent gets stuck in a loop and spends \$500 in 10 minutes.
- **The Solution:**
 - **Hard Limits:** Set a `max_tokens` limit per request and a `max_iterations` limit per agent loop.
 - **Tiering:** Use cheaper models (GPT-4o-mini) for simple routing/summarization and expensive models (GPT-4o/Claude 3.5 Sonnet) only for complex reasoning.
 - **Alerting:** Set up real-time dashboards to track "Cost per User Session."

4. Latency Optimization

- **The Problem:** Users hate waiting 30 seconds for a response.
- **The Solution:**
 - **Streaming:** Stream the text to the UI so the user sees "activity" immediately.
 - **Parallelization:** If an agent needs to call three tools that don't depend on each other, call them concurrently (Async) rather than one by one.
 - **Speculative Execution:** Start a low-latency model to give a "preliminary" answer while the heavy model is still thinking.

5. Rate Limits & Quotas

- **The Problem:** You hit OpenAI's or Anthropic's rate limits (TPM/RPM), and your entire app goes down.
- **The Solution:**
 - **Exponential Backoff:** If you get a 429 error (Rate Limit), wait 1 second, then 2, then 4, before retrying.
 - **Model Fallbacks:** If GPT-4o is rate-limited, automatically failover to Claude 3.5 or a local Llama-3 instance.
 - **Load Balancing:** Distribute requests across multiple API keys or providers.

6. Guardrails & Safety Checks

- **The Problem:** The "Jailbreak" (e.g., the user tricks your meal planner into giving advice on how to make explosives) or "PII Leakage" (the agent accidentally shares another user's address).
- **The Solution:** Use a "Guardrail" layer (like **NeMo Guardrails** or **Guardrails AI**).
 - **Input Guardrails:** Check if the user's prompt is malicious *before* it hits the LLM.
 - **Output Guardrails:** Validate that the LLM's response doesn't contain forbidden topics, profanity, or sensitive data.
 - **Hallucination Checks:** Cross-reference the LLM output against the retrieved documents (RAG) to ensure factuality.

The "Senior" Interview Answer:

"In my last project, I didn't just deploy an agent; I deployed an **observability stack**. We used **semantic caching** to reduce costs by 30% for repeat queries and implemented **circuit breakers** so that if our primary model hit a rate limit, the system gracefully degraded to a faster, cheaper model to maintain 99.9% uptime."

-